



۸۱۰۱۰۱۵۵۸

پردازش اطلاعات کوانتومی  
نام و نام خانوادگی: مهدی وجهی



پروژه ۴

## ۱ مسئله

ما می خواهیم با روش  $VQLS$  مسئله دستگاه معادلات خطی را حل کنیم. در حل این دستگاه ما باید مقدار بردار  $x$  را به دست بیاوریم.

$$Ax = b$$

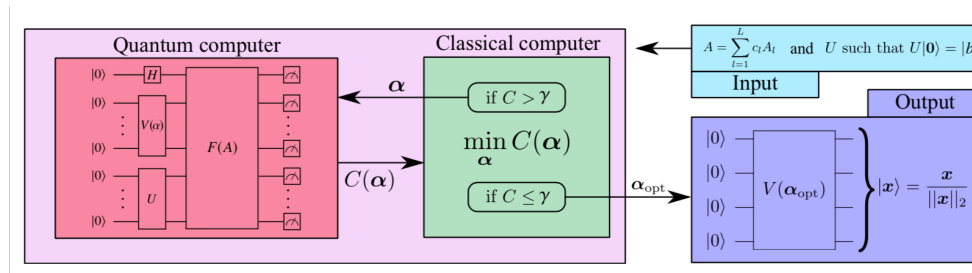
در مقیاس های بزرگ کامپیوتر های کلاسیک قادر به حل آن نیستند و باید به سراغ روش های کوانتومی روش هر چند که روش های بهتری مانند  $HHL$  وجود دارد اما برخلاف آن روش، روش مورد استفاده ما در کامپیوتر های کوانتومی فعلی هم کار می کند.

## ۲ شرح کلی الگوریتم

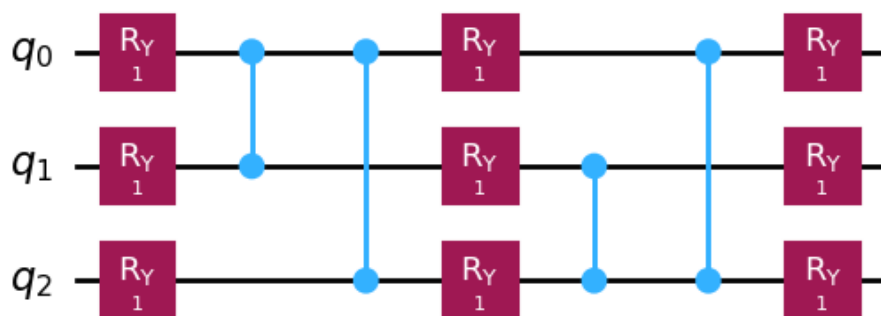
ما باید به روش  $A$  تبدیل به ماتریس یونیتاری کنیم که بتوانیم در فضای کوانتومی با عملگر ها آن را پیاده کنیم. برای این کار کفایت ماتریس را به صورت ترکیب خطی عملگر های یونیتاری بنویسیم.

$$A = \sum_n c_n A_n$$

همچنین برای  $b$  هم عملگری پیدا می کنیم که  $U|0\rangle = |b\rangle$ . کلیت این روش این است که ما با اجرا چندین باره مدار مقدار  $x$  و استفاده از تکنیک های کلاسیک مقدار را تخمین بزنیم.



شکل ۱: نمای کلی روش



شکل ۲: مدار Ansatz

### ۳ مدار Ansatz

به صورت دقیق تر ما مداری با پارامتر  $\alpha$  تعریف می کنیم که خروجی آن  $|\psi(\alpha)\rangle$  است و می خواهیم با آن مقدار نهایی  $|x\rangle$  را تخمین بزنیم به این صورت که توقع داریم با ضرب این مقدار در  $A$  به بردار  $b$  نزدیک شویم. این مقدار را به صورت زیر تعریف می کنیم:

$$|\Phi\rangle = A|\psi(\alpha)\rangle$$

در حالت مطلوب داریم  $|\Phi\rangle = |b\rangle$ . حال تابع هزینه خود را تعریف کنیم. برای ساخت مدار آن به این صورت عمل می کنیم که با پارامترهای تعریف شده چرخش در راستای  $y$  اعمال می کنیم. سه بار این کار را با پارامترهای مختلف انجام می دهیم و بین هر دو تای آن درهم تنیدگی با  $cz$  ایجاد می کنیم.

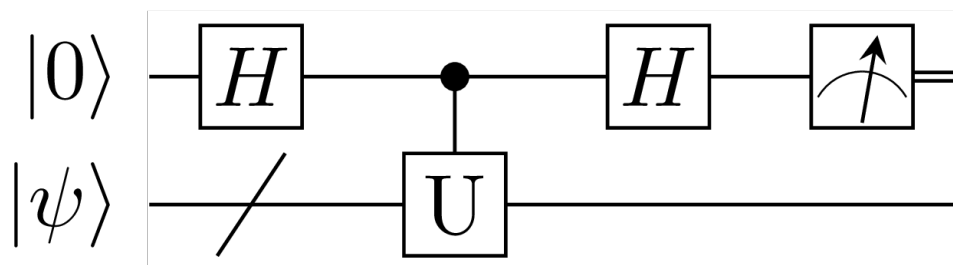
### ۴ تابع هزینه

ما به این صورت عمل می کنیم که اختلاف اندازه بردار فی را با اندازه تصویر را روی  $b$  می گیریم و سعی می کنیم آن را کمینه کنیم. در واقع داریم:

$$H_P = I - |b\rangle\langle b| \quad C_P = \langle\Phi|H_P|\Phi\rangle = \langle\Phi|(I - |b\rangle\langle b|)|\Phi\rangle = \langle\Phi|\Phi\rangle - \langle\Phi|b\rangle\langle b|\Phi\rangle = \langle\Phi|\Phi\rangle - |\langle b|\Phi\rangle|^2$$

تابع هزینه بالا یک مشکل دارد که با کم شدن اندازه بردار کوچک می شود در صورتی که هدف ما این نیست. کافیست که آن را نرمال کنیم.

$$C_P = 1 - \frac{|\langle b|\Phi\rangle|}{\langle\Phi|\Phi\rangle}$$



شکل ۳: مدار تست هادامارد

## ۵ تست هادامارد

حالا برای محاسبه این مقدار از تست هادامارد استفاده می کنیم به این صورت که یک کیوبیت کمکی را در حالت + می بریم و سپس  $U$  را به صورت کنترلی روی  $|\psi\rangle$  اعمال می کنیم و مجدد نتیجه را از هادامارد رد می کنیم و اندازه می گیریم. حال معادله را بررسی کنیم:

$$\begin{aligned} \frac{|0\rangle + |1\rangle}{\sqrt{2}} \otimes |\psi\rangle &= \frac{|0\rangle \otimes |\psi\rangle + |1\rangle \otimes |\psi\rangle}{\sqrt{2}} \\ \frac{|0\rangle \otimes |\psi\rangle + |1\rangle \otimes |\psi\rangle}{\sqrt{2}} &\rightarrow \frac{|0\rangle \otimes |\psi\rangle + |1\rangle \otimes U|\psi\rangle}{\sqrt{2}} \\ \frac{|0\rangle \otimes |\psi\rangle + |1\rangle \otimes U|\psi\rangle}{\sqrt{2}} &\rightarrow \frac{1}{2} [|0\rangle \otimes |\psi\rangle + |1\rangle \otimes |\psi\rangle + |0\rangle \otimes U|\psi\rangle - |1\rangle \otimes U|\psi\rangle] \\ &\Rightarrow \frac{1}{2} |0\rangle \otimes (\mathbb{I} + U)|\psi\rangle + \frac{1}{2} |1\rangle \otimes (\mathbb{I} - U)|\psi\rangle \end{aligned}$$

در این مرحله احتمال وقوع ۲ حالت را بررسی می کنیم:

$$\begin{aligned} P(0) &= \frac{1}{4} \langle \psi | (\mathbb{I} + U)(\mathbb{I} + U^\dagger) | \psi \rangle = \frac{1}{4} \langle \psi | (\mathbb{I}^2 + U + U^\dagger + UU^\dagger) | \psi \rangle = \frac{1}{4} \langle \psi | (2\mathbb{I} + U + U^\dagger) | \psi \rangle \\ &\Rightarrow \frac{1}{4} [2 + \langle \psi | U^\dagger | \psi \rangle + \langle \psi | U | \psi \rangle] = \frac{1}{4} [2 + (\langle \psi | U | \psi \rangle)^* + \langle \psi | U | \psi \rangle] = \frac{1}{2} (1 + \text{Re} \langle \psi | U | \psi \rangle) \end{aligned}$$

به همین صورت برای ۱ داریم:

$$P(1) = \frac{1}{2} (1 - \text{Re} \langle \psi | U | \psi \rangle)$$

در گام آخر اختلاف آنها را محاسبه می کنیم:

$$P(0) - P(1) = \text{Re} \langle \psi | U | \psi \rangle$$

حالا می توانیم یک گام جلو برویم.

## ۶ محاسبه مخرج کسر تابع هزینه

مخرج کسر خطا را باز می کنیم:

$$\begin{aligned} \langle \Phi | \Phi \rangle &= \langle \psi(k) | A^\dagger A | \psi(k) \rangle = \langle 0 | V(k)^\dagger A^\dagger A V(k) | 0 \rangle = \langle 0 | V(k)^\dagger \left( \sum_n c_n A_n \right)^\dagger \left( \sum_n c_n A_n \right) V(k) | 0 \rangle \\ \Rightarrow \langle \Phi | \Phi \rangle &= \sum_m \sum_n c_m^* c_n \langle 0 | V(k)^\dagger \underbrace{A_m^\dagger A_n}_U V(k) | 0 \rangle \end{aligned}$$

حال لازم است تک تک ترم های این عبارت را با همان آزمون هادامارد که صحبت شد محاسبه کنیم. توجه کنید که تمامی مقادیر با توجه به گیت های اعمال شده حقیقی هستند و تنها بخش حقیقی کفایت می کند. پس تا اینجا کار ریاضیات مخرج کسر را متوجه شدیم. در کد ابتدا  $|\psi(k)\rangle$  که همان  $V(k)|0\rangle$  است را محاسبه می کنیم:

```
1 apply_fixed_ansatz(qubits, parameters)
```

با عبور کیوبیت کمکی از هادامارد به ترتیب عملگر های  $A_n$  و  $A_m^\dagger$  را اعمال می کنیم:

```
1 for ie in range(0, len(gate_type[0])):
2     if (gate_type[0][ie] == 1):
3         circ.cz(auxiliary_index, qubits[ie])
4 for ie in range(0, len(gate_type[1])):
5     if (gate_type[1][ie] == 1):
6         circ.cz(auxiliary_index, qubits[ie])
```

در آخر هم مجدد روی کیوبیت کمکی هادامارد اعمال می کنیم.

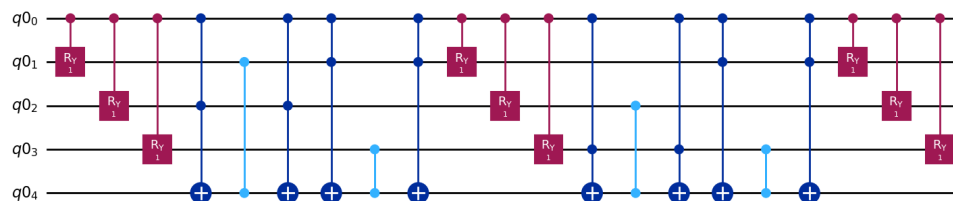
## ۷ محاسبه صورت کسر تابع هزینه

معادله را باز می کنیم:

$$|\langle b | \Phi \rangle|^2 = |\langle b | A V(k) | 0 \rangle|^2 = |\langle 0 | U^\dagger A V(k) | 0 \rangle|^2 = \langle 0 | U^\dagger A V(k) | 0 \rangle \langle 0 | V(k)^\dagger A^\dagger U | 0 \rangle$$

حال  $A$  را باز می کنیم:

$$\langle 0 | U^\dagger A V(k) | 0 \rangle^2 = \sum_m \sum_n c_m^* c_n \langle 0 | U^\dagger A_n V(k) | 0 \rangle \langle 0 | V(k)^\dagger A_m^\dagger U | 0 \rangle$$



شکل ۴: نمونه مدار  $Ansatz$  به صورت کنترلی

چون مقادیر حقیقی است خود آن با دگر آن برابر است پس داریم:

$$|\langle b|\Phi\rangle|^2 = \sum_m \sum_n c_m c_n \langle 0|U^\dagger A_n V(k)|0\rangle \langle 0|U^\dagger A_m V(k)|0\rangle$$

در کد نیز دقیقاً همین را پیاده می‌کنیم. ابتدا برای اعمال  $V(k)$  داریم:

```
1 for i in range (0, len(qubits)):
2     circ.cry(parameters[0][i], qiskit.circuit.Qubit(reg, auxiliary), qiskit.circuit.Qubit(reg, qubits[i]
    )))
```

سپس باید  $ccz$  را اعمال کنیم اما چون همچنین گیتی نداریم مجبوریم با ۳ گیت و یک کیوبیت کمکی آن را اعمال کنیم. مثلاً برای اعمال آن روی کیوبیت ۱ داریم:

```
1 circ.ccx(auxiliary, qubits[1], 4)
2 circ.cz(qubits[0], 4)
3 circ.ccx(auxiliary, qubits[1], 4)
```

و مانند همان بخش قبل که داشتیم این را تکرار می کنیم. در عبارت ریاضی عملگر  $U$  مربوط به  $b$  مانده که هنوز اعمال نشده کافیس است آن را به صورت کنترلی اعمال کنیم. فرض کرده که بردار  $b$  را می توانیم با اعمال هادامارد روی هر کیوبیت ایجاد کنیم. کد آن ساده است و از آن عبور می کنیم.

الان تمام قطعات کامل است و آنها را کنار هم قرار می دهیم:

```
1 def special_had_test(gate_type, qubits, auxiliary_index, parameters, reg):
2
3     circ.h(auxiliary_index)
4
5     control_fixed_ansatz(qubits, parameters, auxiliary_index, reg)
6
7     for ty in range(0, len(gate_type)):
```

```

8         if (gate_type[ty] == 1):
9             circ.cz(auxiliary_index, qubits[ty])
10
11         control_b(auxiliary_index, qubits)
12
13         circ.h(auxiliary_index)
    
```

در ابتدا و انتها گیت هادامارد برای کیوبیت کمکی سپس محاسبه  $V(k)$  در گام بعد محاسبه  $A_n$  با اعمال حلقه مربوطه و سپس اعمال عملگر  $U$  مربوط به ایجاد  $b$ .

## ۸ پیاده سازی تابع هزینه

حال که تمام قطعات پازل را جمع کردیم آنها را کنار هم قرار می دهیم. همانطور که صحبت شد هم برای صورت و هم برای مخرج باید تمام ترکیب های  $A_n, A_m$  را محاسبه کنیم پس دو حلقه می زنیم. در ابتدای آن مدار را تعریف می کنیم و شبیه ساز:

```

1 for i in range(0, len(gate_set)):
2     for j in range(0, len(gate_set)):
3         qctl = QuantumRegister(5)
4         qc = ClassicalRegister(5)
5         circ = QuantumCircuit(qctl, qc)
6         backend = Aer.get_backend('aer_simulator')
    
```

سپس ضریب مربوط به این دو  $A$  را ضرب و محاسبه می کنیم:

```

1 multiply = coefficient_set[i]*coefficient_set[j]
    
```

در گام بعدی از تابعی که مخرج را ایجاد می کند استفاده می کنیم و آن را می سازیم. سپس آن را به گیت های پایه تبدیل می کنیم و تبدیل به فرمت کوانتومی می کنیم هرچند که این کار دیگر در نسخه های جدید لازم نیست و سپس آن را اجرا می کنیم. در آخر وضعیت کیوبیت هارا می خوانیم.

```

1 had_test([gate_set[i], gate_set[j]], [1, 2, 3], 0, parameters)
2 circ.save_statevector()
3 t_circ = transpile(circ, backend)
4 qobj = assemble(t_circ)
5 job = backend.run(qobj)
    
```

```

6 result = job.result()
7 outputstate = np.real(result.get_statevector(circ, decimals=100))
8 o = outputstate
    
```

در گام آخر روی حالات پیمایش می کنیم و حالاتی که کیوبیت اول آنها ۱ است را انتخاب می کنیم و احتمال رخداد آنها را جمع می زنیم.

```

1 m_sum = 0
2 for l in range (0, len(o)):
3     if (l%2 == 1):
4         n = o[l]**2
5         m_sum+=n
    
```

بعد از این عملیات کافیت عبارت زیر را حساب کنیم:

$$Re \langle \psi | U | \psi \rangle = P(0) - P(1) = 1 - 2P(1)$$

که برای هر ضریب آن را حساب می کنیم و جمع می کنیم:

```

1 overall_sum_1+=multiply*(1-(2*m_sum))
    
```

حال به سراغ صورت کسر می رویم. این قسمت کاملاً با قسمت قبل مشابهت دارد تنها همانطور که دی فرمول مشکل است به ازای هر دو مقدار  $A_n, A_m$  نتایج باید در یک دیگر ضرب شوند. برای این کار یک بار به ازای  $A_n$  و بار دیگر به ازای  $A_m$  محاسبه می کنیم و نتایج را ضرب می کنیم و سپس مانند قبل جمع می زنیم. این قسمت کد به شکل زیر است:

```

1 for extra in range(0, 2):
2     ...
3     if (extra == 0):
4         special_had_test(gate_set[i], [1, 2, 3], 0, parameters, qctl)
5     if (extra == 1):
6         special_had_test(gate_set[j], [1, 2, 3], 0, parameters, qctl)
7     ...
8     m_sum = 0
9     for l in range (0, len(o)):
10         if (l%2 == 1):
11             n = o[l]**2
    
```

```

12         m_sum+=n
13     mult = mult*(1-(2*m_sum))

```

حال که صرت و مخرج کسر را داریم در فرمول قرار می دهیم و محاسبه می کنیم:

```

1 1-float(overall_sum_2/overall_sum_1)

```

## ۹ شبیه سازی

حال که تمامی بخش ها کامل شد نوبت شبیه سازی است در این قسمت تنها کافیت معادله خود را تعریف کنیم و روی تابع هزینه که طراحی کردیم یک بهینه ساز کلاسیک صدا بزنیم.

```

1 coefficient_set = [0.55, 0.45]
2 gate_set = [[0, 0, 0], [0, 0, 1]]
3
4 out = minimize(calculate_cost_function, x0=[float(random.randint(0,3000))/1000 for i in range(0, 9)],
5               method="COBYLA", options={'maxiter':200})
6
7 print(out)
8
9 out_f = [out['x'][0:3], out['x'][3:6], out['x'][6:9]]

```

سپس با همین مقدار به دست آمده مقدار بردار  $b$  را حساب می کنیم و با مقدار اصلی شباهت می گیریم:

```

1 circ = QuantumCircuit(3, 3)
2 apply_fixed_ansatz([0, 1, 2], out_f)
3 circ.save_statevector()
4
5 backend = Aer.get_backend('aer_simulator')
6 t_circ = transpile(circ, backend)
7 qobj = assemble(t_circ)
8 job = backend.run(qobj)
9
10 result = job.result()
11 o = result.get_statevector(circ, decimals=10)
12
13 a1 = coefficient_set[1]*np.array([[1,0,0,0,0,0,0,0], [0,1,0,0,0,0,0,0], [0,0,1,0,0,0,0,0],
14                                   [0,0,0,1,0,0,0,0], [0,0,0,0,-1,0,0,0], [0,0,0,0,0,-1,0,0], [0,0,0,0,0,0,-1,0], [0,0,0,0,0,0,0,-1]])

```



```

14 a2 = coefficient_set[0]*np.array([[1,0,0,0,0,0,0,0], [0,1,0,0,0,0,0,0], [0,0,1,0,0,0,0,0],
    [0,0,0,1,0,0,0,0], [0,0,0,0,1,0,0,0], [0,0,0,0,0,1,0,0], [0,0,0,0,0,0,1,0], [0,0,0,0,0,0,0,1]])
15 a3 = np.add(a1, a2)
16
17 b = np.array([float(1/np.sqrt(8)),float(1/np.sqrt(8)),float(1/np.sqrt(8)),float(1/np.sqrt(8)),float(1/np
    .sqrt(8)),float(1/np.sqrt(8)),float(1/np.sqrt(8)),float(1/np.sqrt(8))])
18
19 print((b.dot(a3.dot(o))/(np.linalg.norm(a3.dot(o))))**2)

```

بعنوان نمونه برای نتیجه داریم:

```

1      fun: 0.03302579556573393
2      maxcv: 0.0
3 message: 'Maximum number of function evaluations has been exceeded.'
4      nfev: 200
5      status: 2
6 success: False
7      x: array([ 3.19103414,  2.34621137,  1.10649596, -1.09328699,  1.60020466,
8              1.07590907, -0.18699078,  2.74374942,  2.93056934])
9 (0.966974204438782+0j)

```

که تقریباً ۳ درصد خطا دارد که به نظر مناسب است.

تنها یک نکته مانده آن هم این که ما به جای این که نمونه برداری کنیم مستقیم بردار ها را خواندیم. در حالت واقعی این کار ممکن نیست. برای این کار کافیت کد خود را به گونه ای اصلاح کنیم که نمونه گیری کند و جواب ها را بشمارد و احتمالات را تخمین بزند.

```

1 t_circ = transpile(circ, backend)
2 qobj = assemble(t_circ, shots=10000)
3 job = backend.run(qobj)
4 result = job.result()
5 outputstate = result.get_counts(circ)
6 if ('1' in outputstate.keys()):
7     m_sum = float(outputstate["1"])/100000
8 else:
9     m_sum = 0

```

اگر این کار را با ۱۰۰۰ نمونه برداری انجام دهیم دقت تقریباً ۷۵ درصد می شود که کاهش قابل توجهی است ولی می توان با افزایش نمونه برداری این مشکل را برطرف کرد.